

Computer Programming

Unit 2: Scanners

Contents

2.1 The Role of the Scanner

2.2 Recognizing Words

2.2.1 A Formalism of Recognizer

2.2.2 Recognizing Complex Words

2.3 Regular Expression

2.3.1 Formalizing the Notation

2.3.2 Closure properties of REs

2.4 From Regular Expression to Scanner

2.4.1 Nondeterministic Finite Automata

2.4.2 Regular Expression to NFA: Thomson's Construction

2.4.3 NFA to DFA

2.4.4 Using a DFA as a Recognizer

Scanner (Lexical Analysis)

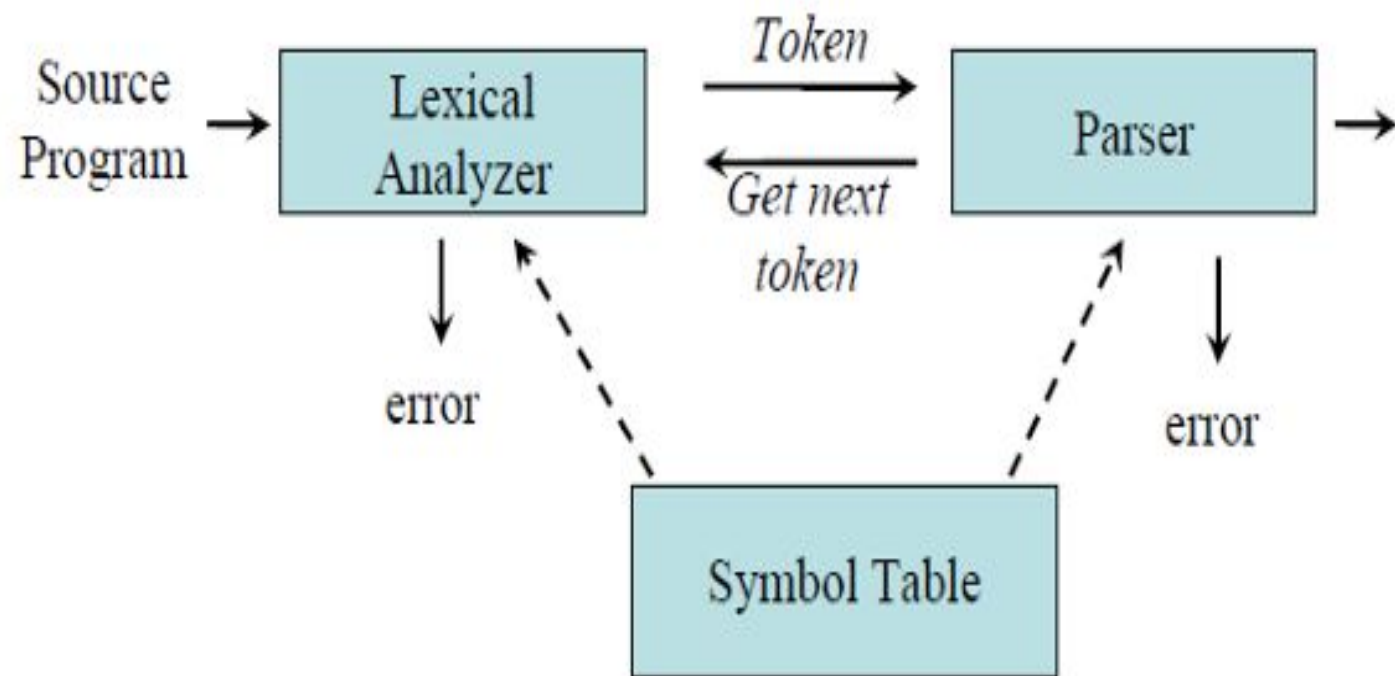
- **Breaks** the **source code** into meaningful **sequences** called **tokens**, by removing any whitespace or comments in the source code.

Tokens are meaningful **sequences of characters**

that can be treated as a **unit in the grammar** of the **programming languages**

such as

keywords/type (int), **identifiers** (a, sum, avg), **operators** (=, +, -), **literals** (fixed/exact value – 10,), and **punctuation** (; ,).



The Role of the Scanner

- is the first phase of the compilation process.
- Its primary **role** is to **read the source code** character by character and **group** them into meaningful **sequences called tokens**.
- These tokens are then passed to the parser (the next phase) for syntax analysis.

Key Functions of a Scanner

1. Reading Input Characters

- The scanner **reads the source program** as a **stream of characters**.
- It ignores unnecessary whitespace, comments, and other irrelevant elements.

2. Tokenization (Lexical Analysis)

- Converts sequences of characters into **tokens** (smallest meaningful units).

Eg, `int x = 10 + y;`

Tokens: `[int, x, =, 10, +, y, ;]`

3. Classification of Tokens

- Each token is associated with a token type (e.g., identifier, keyword, operator, constant).

Eg,

int $\rightarrow$ keyword

x $\rightarrow$ identifier

= $\rightarrow$ assignment operator

10 $\rightarrow$ integer constant

4. Handling Lexical Errors

- Detects invalid tokens (e.g., @xyz, 123abc).
- Reports errors like "Illegal character" or "Unterminated string."

Eg, int a = 10 @ 5;

@ is not a valid operator in C/C++ → **lexical error**

eg, int 1abc = 5;

identifier cannot start with a number → **lexical error**

eg, cout << "Hello;

Missing closing " → **lexical error**

5. Symbol Table Interaction

- Stores identifiers and constants in the symbol table for later phases.

Eg,

```
int age = 20;
```

```
float salary = 5000.50;
```

```
char grade = 'A';
```

Symbol Table Entries

Name	Type	Value
age	int	20
salary	float	5000.50
grade	char	'A'

Recognizing Words

- I. A Formalism of Recognizer
- II. Recognizing Complex Words

What is a “word”?

- A “**word**” is usually a **token**—a basic building block of the source code.

Examples:

- int, while, for, if → keywords
- a, sum → identifiers
- 123, 45.6 → numbers
- +, = → operators

In lexical analysis, **recognizing words** means identifying **meaningful sequences of characters from the input source code** —

these sequences are called **lexemes**, and they are matched against **patterns** (rules) defined for **tokens**.

I. A Formalism of Recognizer

1. Recognizer: Formal Definition

- is a system (or program) that **reads an input string** and **decides** whether the **string belongs to a particular language** or not.

- **Formally:**

A recognizer is a mechanism that **accepts** all **strings** that are part of a **language L** and **rejects** all strings **not in L**.

Eg,

- Language $L = \{ \text{all strings consisting of only 'a' and 'b', ending with 'a' } \}$
- Recognizer will accept 'a', 'ba', 'bba', but reject 'ab', 'bb'.

Simple Working:

- Read the input one symbol at a time (or character by character)
- Transition between different states based on the symbol.
- Decide ACCEPT or REJECT at the end.

2. Alphabets, Strings, and Languages

i. Alphabet (Σ)

- An Alphabet is a **finite set of symbols**.
- It is the **building block** for strings
- Denoted by Σ (sigma).

Eg,

$\Sigma = \{ 0, 1 \}$ (binary alphabet)

$\Sigma = \{ a, b \}$ (letters)

means Only two symbols exist: a and b

- **Key idea** : Σ contains **characters / symbols**

(Note: Even though 0 and 1 look like numbers:

*In formal languages, they are treated as **characters (symbols)**, not numeric values)*

Finite- limited or countable

ii. String

- A String is a **finite sequence of symbols chosen** from an **alphabet**.

Eg,

If $\Sigma = \{ a, b \}$,

then the are strings over Σ are:

"a", "b", "ab", "ba", "aa", "bb", "aab", "aba", "aaba"

.

*(A **string over Σ** means any finite sequence of symbols taken from Σ)*

iii. Language

- A Language is a **set of strings**

Strings are made using **symbols** from alphabet Σ .

Eg,

If $\Sigma = \{ a, b \}$,

A possible language L could be:

$L = \{ a, b, ab, ba, aab, aaba, ba, bba \}$

Eg,

If $\Sigma = \{ 0, 1 \}$,

A possible language L could be:

$L = \{ \text{strings with an even number of 0's} \}$

i.e. 00, 100, 0011, 11100, 100001 ✓

10, 10001, ... ✗

3. Simple Word Recognition

- Recognizes **one OR many** characters forming a **valid word/token**

One-character words : a, 0

Multi-character words : aaa, sum, 1010

Example 1: Matching Identifiers

- In programming, an identifier usually follows **rules** like:
 - Starts with a letter (a–z, A–Z) or underscore (_)
 - Followed by letters or digits (a–z, A–Z, 0–9)
- **Regular Expression** for an identifier:

`[a-zA-Z][a-zA-Z0-9]*`

Or, `[_a-zA-Z][a-zA-Z0-9_]*`

Eg,

sum, a, total1, X12 ✓

1total, %abc (illegal symbol) ✗

where,

$[a-zA-Z] \rightarrow$ First character must be a **letter**

- any one lowercase letter or uppercase letter

(eg, a, B)

( aa, ab, aB1- more than 1 character and contains digit)

$[a-zA-Z0-9]^* \rightarrow$ Followed by **zero or more occurrences of letters or digits**

i.e. means **0 or more repetitions**

(eg, abc, Bad, a1b2, 123, " "- empty string)

Example 2: Recognizing Numbers

- **Simple Numbers** are recognized by all **valid non-negative integers**
i.e. 0-9

Regular Expression for numbers:

$[0-9]^+$

$[0-9] \rightarrow$ any digit,

$^+ \rightarrow$ one or more occurrences of digits

Eg,

0, 123, 4567 ✓

12a3 ✗

Regular Expression for

Integer recognition : $[0-9]^+$

Floating-point number recognition: $[0-9]^+\backslash.[0-9]^+$

Signed numbers : $[+-]?[0-9]^+$

Signed Floating Point Number: $[+-]?[0-9]^+\backslash.[0-9]^+$

Eg,

0, 123, 4567 ✓

12a3 ✗

5.6 ✓

-10, +45, -5.5 ✓

$[0-9]^+\backslash.[0-9]^+$: $[0-9]^+ \rightarrow$ one or more digits (before decimal)

$\backslash. \rightarrow$ a real dot (.)

$[0-9]^+ \rightarrow$ one or more digits (after decimal)

. alone = any character
 $\backslash.$ = actual **dot** (.)

$[+-]? \rightarrow$ optional + or – $\square$ **+ or – (may appear once or may not appear)**

In regular expressions: ? means “optional (0 or 1 occurrence only not more than 1)”

Operator	Meaning	Empty string allowed?
*	0 or more occurrences	✓ Yes
+	1 or more occurrences	✗ No
?	0 or 1 occurrence only	✓ Yes

Eg for a*:

"" ✓ (empty string)
a ✓
aa ✓
aaa ✓

Eg for a+:

"" ✗ (not allowed)
a ✓
aa ✓
aaa ✓

Recognizing Complex Words

a scanner must **recognize complex patterns** such as:

- Keywords (e.g., int, if, else, while, return)
- Identifiers (e.g., total, count1)
- Numbers (e.g., 123, 3.14)
- Operators (e.g., +, -, ==, >=)
- Punctuation (e.g., ;, {, })

Recognizing these complex words needs careful design of patterns and rules for matching.

Recognizing Complex Patterns

- Each type of **complex word** is recognized by **defining a regular expression** (RE) or rule for it.

Token Type	Pattern	Regular Expressions
Identifiers	Letter followed by letters/digits	[a-zA-Z][a-zA-Z0-9]*
Keywords	Exact match strings	if, while, return
Numbers (Integers)	One or more digits	[0-9]+
Numbers (Real)	Digits + dot + digits	[0-9]+\.[0-9]+ (eg, 12.34)
Operators	Single or double character patterns	+, -, ==, >=
Punctuation	Exact single characters	(,), {, }, ;

Regular Expression

I. Formalizing the Notation

II. Closure properties of REs

Regular Expressions (RE)

- A regular expression **defines a pattern for a set of strings**, and that **set of strings** is called a **language**.

So:

- RE $\rightarrow$ defines patterns
- Pattern $\rightarrow$ set of strings
- Those patterns $\rightarrow$ produce a **language (set of strings)**

Formally,

- are **symbolic notations** that are used to **specify patterns for strings** of a **language**.

i.e. identify **valid string/token/lexeme** over an alphabet (Σ).

- Each **pattern** matches a set of **strings**.

Pattern - $1(1+0)^*0$

Eg, Given the alphabet $A = \{0, 1\}$

- $1(1+0)^*0$

denotes the language of all string that begins with a '1' and ends with a '0'.

i.e. Generated language is:

$L = \{\text{all string over } \{0,1\} \text{ that begins with a 1 and ends with a 0}\}$

Eg, 10, 110, 100, 1010, 1000, 10010, 11100010....

$(1+0)$ - means **either 1 or 0 or none**, So it represents the set: $\{0, 1\}$

$*$ - means **0 or more repetitions**

- $(1+0)^*00$

$L = \{\text{all string over } \{0,1\} \text{ that ends with } 00\}$

eg, 100,000, 110,...

- $1(10)^*0$

$L = \{\text{all binary string that start with } 1, \text{ contain zero or more repetitions of } 10, \text{ and end with } 0\}$

eg, 10, 1100, 11101000, 1111010000

10 - It means **first 1, then 0**

I. Formalizing the Notation of RE

- **defines a precise mathematical syntax and meaning** for patterns used in lexical analysis.
- **Purpose** - To describe how **tokens** (keywords, identifiers, numbers, etc.) are written using a **formal language notation**.

A regular expression (RE) is built using:

1. **Symbols (alphabet Σ):** eg, {a, b, 0, 1}

2. **Empty string (ϵ)** $\rightarrow$ language with no strings

ϵ (**epsilon**) represents a string with **no characters**

eg, $\{\epsilon\}$ - contains one string,

$\{\epsilon, a, 00\}$ - contains three strings

3. **Empty set ($\emptyset$)** $\rightarrow$ language with only empty string

eg, $\{\}$

Number of strings = **0**

4. Operators:

- Union: $a \mid b \rightarrow$ either a or b
- Concatenation: $ab \rightarrow$ a followed by b
- Kleene star: $a^* \rightarrow$ zero or more a 's

Detailly explained in Closure properties of REs

II. Closure properties of REs

- This part is about **what operations keep a language regular.**

i.e. If you apply **certain operations** to regular **languages**, the result is guaranteed to produce **regular language**.

1. Union (U)

- Meaning: **OR**

- Eg,

$$a \mid b \rightarrow \{a, b\}$$

string can follow either path a or b

- *choose one option*

Let L_1 and L_2 be regular languages:

- The **union** of two languages L_1 and L_2 is the set of strings that are in L_1 , or L_2 , or in both.

i.e. either L_1 , or L_2

Mathematically:

$$L_1 \cup L_2 = \{ x \mid x \in L_1 \text{ or } x \in L_2 \}$$

Eg,

If $L_1 = \{ a, b \}$ and $L_2 = \{ b, c \}$,

Then, $L_1 \cup L_2 = \{ a, b, c \}$

$\mid$ -> All elements x such that x is in L_1 or x is in L_2 $\in$ -> belongs to

2. Concatenation ($\cdot$)

- Meaning: **And**

- Eg,
 $ab \rightarrow$ first a, then b (/a followed by b)

- go step-by-step

The **concatenation** of two languages L_1 and L_2 is the set of strings formed by **taking a string from L_1** and **immediately following it with a string from L_2** .

i.e. L_1 followed by L_2

Mathematically:

$$L_1 \cdot L_2 = \{ xy \mid x \in L_1 \text{ and } y \in L_2 \}$$

Eg,

If $L_1 = \{ a, b \}$ and $L_2 = \{ c, d \}$,

Then, $L_1 \cdot L_2 = \{ ac, ad, bc, bd \}$

3. Kleene Star (*)

- Meaning: **0 or more repetitions**

- Eg,

$$a^* \rightarrow \{ \epsilon, a, aa, aaa, \dots \}$$

- *Always includes **ϵ (empty string)***

The **Kleene star** of a language L is the **set of all strings** formed by **concatenating zero or more strings** from L .

Mathematically:

$$L^* = \{ \varepsilon \} \cup L \cup (L \cdot L) \cup (L \cdot L \cdot L) \cup \dots$$

(where ε = empty string)

Eg, If $L = \{ a \}$,

Then, $L^* = \{ \varepsilon, a, aa, aaa, aaaa, \dots \}$

4. Kleene Plus (+)

- The Kleene plus of a language L is the set of **all strings formed by one or more concatenations** of **strings from L** (at least one occurrence).

Mathematically:

$$\begin{aligned} L^+ &= L \cdot L^* \\ &= L \cdot (\{ \epsilon \} \cup L \cup (L \cdot L) \cup (L \cdot L \cdot L) \cup \dots) \\ &= L \cdot \epsilon \cup L \cdot L \cup L \cdot L^2 \cup \dots \\ &= L \cup L^2 \cup L^3 \cup \dots \end{aligned}$$

Eg,

If $L = \{ b \}$,

Then, $L^+ = \{ b, bb, bbb, bbbb, \dots \}$

5. Intersection ($\cap$)

- The intersection of two languages L_1 and L_2 is the **set of strings** that are **common to both L_1 and L_2** .

Mathematically:

$$L_1 \cap L_2 = \{ x \mid x \in L_1 \text{ and } x \in L_2 \}$$

Eg,

If $L_1 = \{ a, b, c \}$ and $L_2 = \{ b, c, d \}$,

Then, $L_1 \cap L_2 = \{ b, c \}$

6. Difference (-)

- The difference of two languages L_1 and L_2 is the **set of strings** that are in **L_1 but not in L_2** .

Mathematically:

$$L_1 - L_2 = \{ x \mid x \in L_1 \text{ and } x \notin L_2 \}$$

Eg,

If $L_1 = \{ a, b, c \}$ and $L_2 = \{ b, d \}$,

Then, $L_1 - L_2 = \{ a, c \}$

7. Complementation

- The complement of a language L (with respect to the universal set Σ^*) is the **set of strings not in L** .

Mathematically:

$$L' = \Sigma^* - L$$

Eg,

If $\Sigma = \{ a, b \}$ and $L = \{ \text{strings that start with 'a'} \}$,

Then, $L' = \{ \text{all strings that do not start with 'a'} \}$

i.e. $L' = \{ b, ba, bb, bab, baba, \dots \}$

- A regular expression **defines a pattern for a set of strings**

Regular Grammar

- A regular grammar defines a **set of rules** that helps to generate a **set of strings**.
and That **set of strings** is called a **language**.

So,

- Regular grammar $\rightarrow$ gives rules (i.e. production rules eg, $S \rightarrow aS$)
- Those rules $\rightarrow$ generate strings
- strings together $\rightarrow$ form a language (regular language)

A common **grammar** for **regular expressions** is:

$R \rightarrow R \mid R$

$R \rightarrow R R$

$R \rightarrow R^*$

$R \rightarrow R \cdot R^*$

$R \rightarrow (R)$

$R \rightarrow a$

$R \rightarrow \varepsilon$

Rule	RE	Meaning
$R \rightarrow R \mid R$	$R \mid R$	Union
$R \rightarrow R.R$	$R.R$	Concatenation
$R \rightarrow R^*$	R^*	Kleene star
$R \rightarrow R \cdot R^*$	$R \cdot R^*$	Kleene plus
$R \rightarrow (R)$	(R)	grouping
$R \rightarrow a$	a	symbol from alphabet
$R \rightarrow \varepsilon$	ε	empty string

- The **language** defined by **regular grammar** is known as **regular language**.

Eg,

Regular Expression

a^*b

Meaning: zero or more **a** followed by one **b**

Corresponding Regular Grammar

$S \rightarrow aS$

$S \rightarrow b$

Regular Language Generated

$L = \{ \text{all strings of zero or more } a\text{'s followed by exactly one } b \}$

$= \{ b, ab, aab, aaab, \dots \}$

Eg,

Regular Expression

$(a|b)^*$

Meaning: zero or more **a or b**

Corresponding Regular Grammar

$S \rightarrow aS$

$S \rightarrow bS$

$S \rightarrow \epsilon$

$S \rightarrow a$

$S \rightarrow b$

OR,

$S \rightarrow aS | bS | \epsilon$

$S \rightarrow a$

$S \rightarrow b$

Regular Language Generated

L = all strings of a's and b's including empty string

= $\{\epsilon, a, b, ab, ba, aab, bba, abba, \dots\}$

Summary

Concept	Defines
Regular Expression	are like a search pattern which helps in defining or generating a regular language
Regular Grammar	Are a set of rules (like $A \rightarrow aB$) which generates language
Regular Language	set of strings

A **language** is said to be **regular language** if and only if a **finite state machine** recognizes it.

So what languages are not regular?

□ The languages

- which are not recognized by **finite state machine**
- which require memory
 - as memory of FSM is very limited or (stores only current state)
 - It cant store or count strings